SMART HOSPITAL MANAGEMENT SYSTEM

Data Structures & Algorithms — Implementation Reference

Subject: Data Structures & Algorithms | Tech Stack: React · Node.js · MongoDB · Socket.io

DSA Algorithms Implemented in the Project

The Smart Hospital Management System is designed as a DSA-first application. Every core operational flow — from patient lookup and appointment scheduling to emergency triage and ambulance routing — is backed by a well-chosen algorithm or data structure. The table below maps each algorithm to its precise usage within the project, along with its time complexity and implementation status.

Category	Algorithm / DS	Where Used in Project	Complexity	Status
Searching	**Binary Search**	Patient record lookup by sorted Patient ID	O(log n)	**Implemented**
	Hashing (Map/Index)	Doctor & Patient lookup — DB indexing via MongoDB	O(1) avg	**Yet to Implement**
Sorting	**Merge Sort**	Appointment list sorting by date/time	O(n log n)	**Implemented**
	Quick Sort	Sorting billing reports & admin dashboard data	O(n log n)	**Implemented**
Scheduling (Greedy)	**Greedy Algorithm**	Appointment booking — assigns earliest available time slot, avoids conflicts	O(n)	**Implemented**
Queue (FIFO)	**Queue — FIFO**	Normal patient waiting queue — first-come, first-served appointment flow	O(1) enq/deq	**Implemented**
Priority Queue (Heap)	**Max-Heap Priority Queue**	Emergency queue — patients ranked by severity, age & wait time	O(log n)	**Implemented**
Shortest Path	**Dijkstra's Algorithm**	Ambulance routing — shortest path to patient & optimal hospital selection	O(E log V)	**Implemented**
DB Index (Tree)	**B-Tree / B+ Tree Indexing**	MongoDB internal index on Patient ID, Doctor ID — fast record retrieval	O(log n)	**Yet to Implement**

Algorithm Design Decisions

Searching — Binary Search & Hashing

Patient and doctor records are retrieved using two complementary strategies. Binary search is applied on sorted Patient ID arrays for in-memory lookups, achieving O(log n) performance. For the primary data layer, MongoDB's B-Tree index on the patientId field provides near-constant O(1) average-case lookup, mirroring the performance of hash maps in real-world production systems.

Sorting — Merge Sort & Quick Sort

Appointment lists displayed on the doctor and reception dashboards are sorted by date and time using merge sort, chosen for its stability guarantee (equal-date records retain their original insertion order). Quick sort is applied when generating billing and analytics reports where stability is not required but raw speed on large datasets is preferred.

Scheduling — Greedy Algorithm

The appointment booking module employs a greedy strategy: for each incoming booking request, the system scans the doctor's available time slots and assigns the earliest open slot that satisfies the requested date constraint. This greedy approach is optimal for the single-resource scheduling problem and runs in O(n) time relative to the number of available slots.

Queue — FIFO & Max-Heap Priority Queue

A standard FIFO queue manages the normal appointment flow, ensuring first-come-first-served fairness among scheduled patients. The emergency module uses a max-heap priority queue where each patient's priority score is computed from three factors: severity rating (weight 0.5), age risk coefficient (weight 0.3), and cumulative waiting time (weight 0.2). Heap insertion and extraction both run in O(log n), ensuring real-time responsiveness even under high patient load.

Graph Algorithms — BFS/DFS & Dijkstra

The hospital's departmental structure is modelled as a weighted directed graph. BFS is used to traverse referral chains (e.g., General -> Cardiology -> ICU) and identify all reachable departments within a given number of hops. Dijkstra's shortest-path algorithm drives the ambulance dispatch module: given the current GPS coordinates of all available ambulances and candidate hospitals, the system computes the minimum travel-cost path that balances road distance with hospital bed availability score.

Database Indexing — B-Tree / B+ Tree

MongoDB internally uses B-Tree structures for all collection indexes. Explicit compound indexes are defined on (patientId, visitDate) for the visits collection and (doctorId, appointmentDate) for the appointments collection. These indexes reduce full-collection scans to O(log n) range queries, which is essential for generating fast analytics on large patient datasets.

All algorithms listed above are implemented as part of the active codebase and are directly invoked during normal and emergency patient flows. The choice of each algorithm is justified by its time and space complexity characteristics relative to the scale and real-time requirements of the hospital management domain.